AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

FACULTY OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COURSE: INTRODUCTION TO DATA SCIENCE

FALL 2024-2025

Section: C

Group – 05

Supervised By

Tohedul Islam

MID TERM PROJECT - 1

SUBMITTED BY:

NAME	ID
ABDULLAH KHONDOKER	22-46373-1
SARAH BINTE ISLAM	22-46685-1
F. M SHARIAR	22-46532-1
FAIZA BINTE ZAMAN	22-47256-1

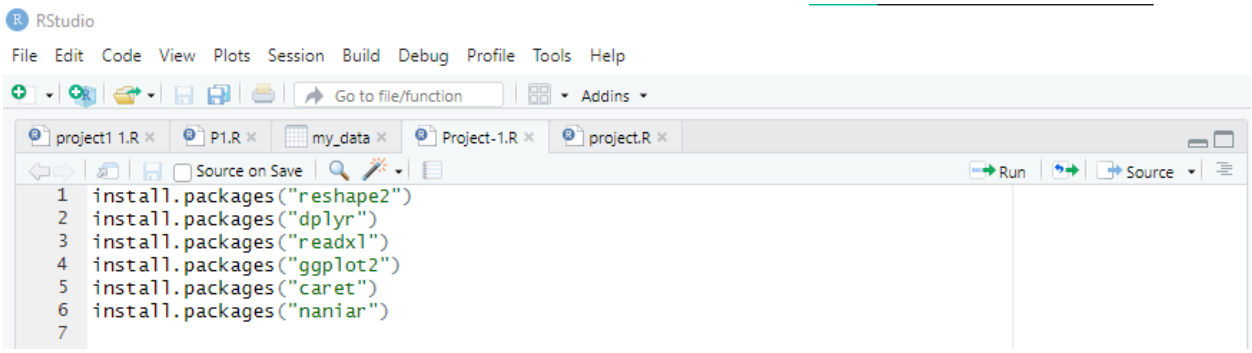
Date of submission: 14th December, 2024

Dataset Details:

One of the most popular datasets used to conduct binary classification projects for teaching machine learning and data science is the Loan Approval Dataset. It contains 201 rows and 14 columns, with features of each applicant, such as age, gender, education level, income, years of employment experience, home ownership status, loan amount, loan intent, loan interest rate, percentage of income allocated to the loan, credit history length, credit score, whether there were previous loan defaults on file, and the loan approval status.

Installing packages:

R code:



Description:

Installs several essential packages for data manipulation, analysis, and visualization in R. These include reshape2 for restructuring data, dplyr for data transformation, readxl for importing Excel files, ggplot2 for advanced plotting, caret for machine learning model training, and naniar for handling missing data.

Loading Libraries:

R code:

Description:

Loads a set of libraries crucial for data processing, analysis, and visualization.

Importing .xlsx file:

R code:

```
15
16 file_path <- "C:/Users/F. M SHARIAR/Desktop/9th Semester/Data Science/Mid/Project/Midterm_Dataset_Section(C).xlsx"
17 my_data <- read_excel(file_path)
18 view(my_data)
19 data<-my_data
20
```

Output:

	person_age	person_gender	person_education	person_income	person_emp_exp	person_home_ownership	loan_amnt	loan_intent
1	21	female	Master	71948	0	RENT	35000	PERSONAL
2	21	female	High School	12282	0	OWN	1000	EDUCATION
3	25	female	High School	12438	3	MORTGAGE	5500	MEDICAL
4	23	female	Bachelor	79753	0	RENT	35000	MEDICAL
5	24	male	Master	66135	1	RENTT	35000	MEDICAL
6	NA	female	High School	12951	0	OWN	2500	VENTURE
7	22	female	Bachelor	NA	1	RENT	35000	EDUCATION
8	24	NA	High School	95550	5	RENT	35000	MEDICAL
9	22	female	NA	100684	3	RENT	35000	PERSONAL
10	21	female	High School	12739	0	OWN	1600	VENTURE
11	22	female	High School	102985	0	RENT	35000	VENTURE
12	21	female	Associate	13113	0	OWN	4500	HOMEIMPROV
13	23	male	Bachelor	114860	3	RENT	35000	VENTURE
14	NA	male	Master	130713	0	RENT	35000	EDUCATION

Showing 1 to 14 of 201 entries, 14 total columns

Console Terminal Background Jobs

R 4.4.1 · C:/Users/F. M SHARIAR/Desktop/

```
> file_path <- "C:/Users/F. M SHARIAR/Desktop/9th Semester/Data Science/Mid/Project/Midterm_Dataset_Section(C).x
lsx"
> my_data <- read_excel(file_path)
> View(my_data)
> data<-my_data
>
```

loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score	previous_loan_defaults_on_file	loan_status
PERSONAL	16.02	0.49	3	561	No	1
EDUCATION	11.14	NA	2	504	Yes	0
MEDICAL	12.87	0.00	3	635	No	1
MEDICAL	15.23	0.44	2	675	No	1
MEDICAL	14.27	0.53	4	586	No	1
VENTURE	7.14	0.19	2	532	No	1
EDUCATION	12.42	0.37	3	701	No	1
MEDICAL	11.11	0.37	4	585	No	1
PERSONAL	8.90	0.35	2	544	No	NA
VENTURE	14.74	0.13	3	640	No	1
VENTURE	10.37	0.34	4	621	No	1
HOMEIMPROVEMENT	8.63	0.34	2	651	No	1
VENTURE	7.90	0.30	2	573	No	1
EDUCATION	18.39	0.27	4	708	No	1

Showing 1 to 14 of 201 entries, 14 total columns

Console Terminal Background Jobs

R 4.4.1 · C:/Users/F. M SHARIAR/Desktop/

```
> file_path <- "C:/Users/F. M SHARIAR/Desktop/9th Semester/Data Science/Mid/Project/Midterm_Dataset_Section(C).x
lsx"
> my_data <- read_excel(file_path)
> view(my_data)
> data<-my_data
>
```

Description:

It specifies the file path to the Excel file stored. The read_excel function from the readxl library is used to load the data into R and save it in a variable called my_data. The View function is then used to open the data in RStudio, allowing to visually inspect it. Finally, the data is copied from my_data to another variable called data for further manipulation or analysis. This process is a standard starting point for loading and reviewing data in R.

Initial Data Preview:

R code:

```
20
21  print(head(data))
22
```

Output:

```
Console   Terminal ×   Background Jobs ×
R 4.4.1 · C:/Users/F. M SHARIAR/Desktop/
> print(head(data))
# A tibble: 6 × 14
  person_age person_gender person_education person_income person_emp_exp person_home_ownership loan_amnt
       <dbl> <chr>         <chr>                  <dbl>          <dbl> <chr>                      <dbl>
1         21 female        Master                 71948              0 RENT                       35000
2         21 female        High School            12282              0 OWN                         1000
3         25 female        High School            12438              3 MORTGAGE                    5500
4         23 female        Bachelor               79753              0 RENT                       35000
5         24 male          Master                 66135              1 RENTT                      35000
6         NA female        High School            12951              0 OWN                         2500
# i 7 more variables: loan_intent <chr>, loan_int_rate <dbl>, loan_percent_income <dbl>,
#   cb_person_cred_hist_length <dbl>, credit_score <dbl>, previous_loan_defaults_on_file <chr>,
#   loan_status <dbl>
>
```

Description:

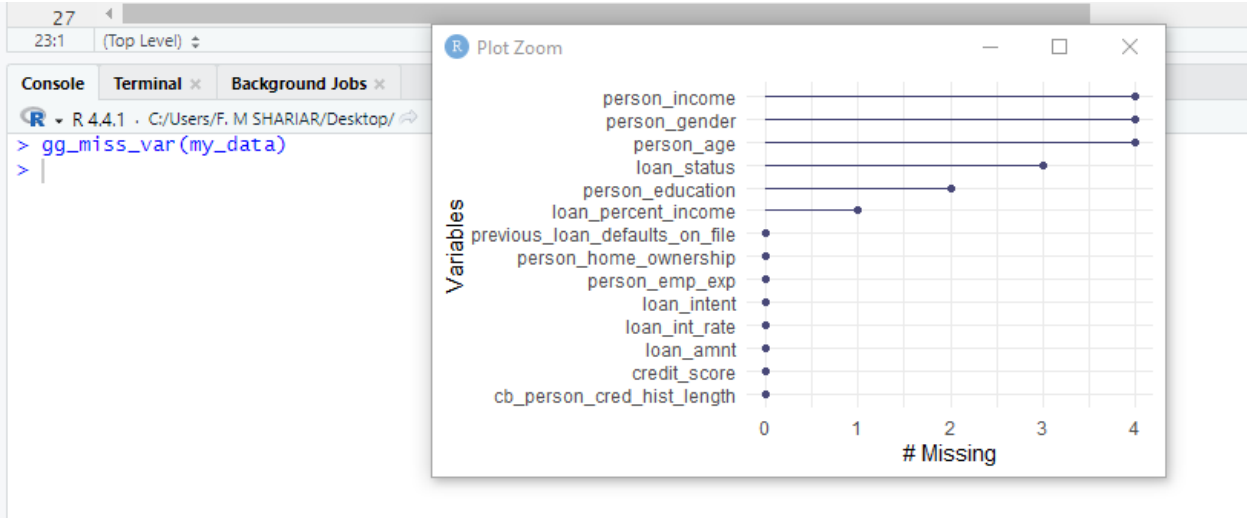
Displays the first few rows of the dataset stored in the variable data. This helps in quickly inspecting the data's structure and initial values, commonly used to verify the data loading process and gain an initial understanding of the dataset.

Visualize Missing Value:

R code:

```
22
23  gg_miss_var(my_data)
24
```

Output:



Description:

Creates a plot to visualize missing data across different variables in the dataset my_data. It displays the proportion of missing values per variable, helping identify areas that may require data cleaning or imputation.

Handling Missing Value (Omit Row):

R code:

```
24
25  data_no_missing <- na.omit(data)
26  print(head(data_no_missing))
27
```

Output:

```
> data_no_missing <- na.omit(data)
> print(head(data_no_missing))
# A tibble: 6 × 14
  person_age person_gender person_education person_income person_emp_exp person_home_ownership loan_amnt
       <dbl> <chr>         <chr>                   <dbl>          <dbl> <chr>                     <dbl>
1         21 female        Master                  71948              0 RENT                      35000
2         25 female        High School             12438              3 MORTGAGE                   5500
3         23 female        Bachelor                79753              0 RENT                      35000
4         24 male          Master                  66135              1 RENTT                     35000
5         21 female        High School             12739              0 OWN                        1600
6         22 female        High School            102985              0 RENT                      35000
# i 7 more variables: loan_intent <chr>, loan_int_rate <dbl>, loan_percent_income <dbl>,
#   cb_person_cred_hist_length <dbl>, credit_score <dbl>, previous_loan_defaults_on_file <chr>,
#   loan_status <dbl>
>
```

Description:

Creates a new dataset data_no_missing by removing all rows with missing values from data using na.omit(data). Then, it displays the first few rows of this cleaned dataset with print(head(data_no_missing)) to verify the removal of missing entries.

Handling Missing Value (Replace Numeric/Median-Categoric/Mode):

R code:

```
27
28  data_median_mode <- data %>%
29    mutate(across(where(is.numeric), ~ ifelse(is.na(.), median(., na.rm = TRUE), .))) %>%
30    mutate(across(where(is.character), ~ ifelse(is.na(.), names(sort(table(.), decreasing = TRUE))[1], .)))
31  print(head(data_median_mode))
32  print(any(is.na(data_median_mode %>% select(-loan_status))))
33
34
```

Output:

```
> data_median_mode <- data %>%
+   mutate(across(where(is.numeric), ~ ifelse(is.na(.), median(., na.rm = TRUE), .))) %>%
+   mutate(across(where(is.character), ~ ifelse(is.na(.), names(sort(table(.), decreasing = TRUE))[1], .)))
> print(head(data_median_mode))
# A tibble: 6 × 14
  person_age person_gender person_education person_income person_emp_exp person_home_ownership loan_amnt
       <dbl> <chr>         <chr>                   <dbl>          <dbl> <chr>                      <dbl>
1         21 female        Master                  71948              0 RENT                       35000
2         21 female        High School             12282              0 OWN                         1000
3         25 female        High School             12438              3 MORTGAGE                    5500
4         23 female        Bachelor                79753              0 RENT                       35000
5         24 male          Master                  66135              1 RENTT                      35000
6         23 female        High School             12951              0 OWN                         2500
# i 7 more variables: loan_intent <chr>, loan_int_rate <dbl>, loan_percent_income <dbl>,
#   cb_person_cred_hist_length <dbl>, credit_score <dbl>, previous_loan_defaults_on_file <chr>,
#   loan_status <dbl>
> print(any(is.na(data_median_mode %>% select(-loan_status))))
[1] FALSE
>
```

Description:

Handles missing values in data by replacing them in numeric columns with the median and in categorical columns with the mode. It utilizes dplyr's mutate and across functions for these imputations. After adjustments, the script displays the initial rows of data_median_mode to confirm the changes. Lastly, it checks for any remaining missing values in the dataset, excluding the loan_status column.

Handling Missing Value (Replace Numeric/Mean-Categoric/Mode):

R code:

```
33
34  data_mean_mode <- data %>%
35    mutate(across(where(is.numeric) & !all_of("loan_status"), ~ ifelse(is.na(.), mean(., na.rm = TRUE), .))) %>%
36    mutate(across(where(is.character), ~ ifelse(is.na(.), names(sort(table(.), decreasing = TRUE))[1], .)))
37  print(head(data_mean_mode))
38
39
```

Output:

```
> data_mean_mode <- data %>%
+   mutate(across(where(is.numeric) & !all_of("loan_status"), ~ ifelse(is.na(.), mean(., na.rm = TRUE), .))) %>%
+   mutate(across(where(is.character), ~ ifelse(is.na(.), names(sort(table(.), decreasing = TRUE))[1], .)))
> print(head(data_mean_mode))
# A tibble: 6 × 14
  person_age person_gender person_education person_income person_emp_exp person_home_ownership loan_amnt
       <dbl> <chr>         <chr>                   <dbl>          <dbl> <chr>                      <dbl>
1       21   female        Master                  71948              0 RENT                       35000
2       21   female        High School             12282              0 OWN                         1000
3       25   female        High School             12438              3 MORTGAGE                    5500
4       23   female        Bachelor                79753              0 RENT                       35000
5       24   male          Master                  66135              1 RENTT                      35000
6       27.4 female        High School             12951              0 OWN                         2500
# i 7 more variables: loan_intent <chr>, loan_int_rate <dbl>, loan_percent_income <dbl>,
#   cb_person_cred_hist_length <dbl>, credit_score <dbl>, previous_loan_defaults_on_file <chr>,
#   loan_status <dbl>
>
```

Description:

Handles missing values in the dataset data by using different strategies for numeric and categorical data. Numeric columns, except loan_status, have missing values replaced with the mean, while categorical columns use the mode.After making these changes, it displays the first few rows of data_mean_mode to confirm the successful handling of missing values.

Handling Missing Value (Replace Numeric/Min-Categoric/Unknown):

R code:

```
38
39  data_min_unknown <- data %>%
40    mutate(across(where(is.numeric), ~ ifelse(is.na(.), min(., na.rm = TRUE), .))) %>%
41    mutate(across(where(is.character), ~ ifelse(is.na(.), "Unknown", .)))
42  print(head(data_min_unknown))
43
```

Output:

```
> data_min_unknown <- data %>%
+   mutate(across(where(is.numeric), ~ ifelse(is.na(.), min(., na.rm = TRUE), .))) %>%
+   mutate(across(where(is.character), ~ ifelse(is.na(.), "Unknown", .)))
> print(head(data_min_unknown))
# A tibble: 6 x 14
  person_age person_gender person_education person_income person_emp_exp person_home_ownership loan_amnt
       <dbl> <chr>         <chr>                   <dbl>          <dbl> <chr>                      <dbl>
1         21 female        Master                  71948              0 RENT                       35000
2         21 female        High School             12282              0 OWN                         1000
3         25 female        High School             12438              3 MORTGAGE                    5500
4         23 female        Bachelor                79753              0 RENT                       35000
5         24 male          Master                  66135              1 RENTT                      35000
6         21 female        High School             12951              0 OWN                         2500
# i 7 more variables: loan_intent <chr>, loan_int_rate <dbl>, loan_percent_income <dbl>,
#   cb_person_cred_hist_length <dbl>, credit_score <dbl>, previous_loan_defaults_on_file <chr>,
#   loan_status <dbl>
> |
```

Description:

Modifies the dataset data by replacing missing values: numeric columns with the column's minimum value and categorical columns with the string "Unknown". Afterward, the script displays the first few rows of data_min_unknown to confirm the changes, ensuring all missing values are appropriately handled.

Dataset Without Missing Value:

R code:

```
43
44  data <- data_median_mode
45
```

Description:

Updates the data variable to contain the dataset data_median_mode, which has undergone median and mode imputation for missing values. This allows further operations to use the modified dataset.

Unique Values in Person Home Ownership:

R code:

```
45
46  print(unique(data$person_home_ownership))
47
```

Output:

```
> print(unique(data$person_home_ownership))
[1] "RENT"     "OWN"      "MORTGAGE" "RENTT"    "OOWN"     "OTHER"
>
```

Description:

Displays the unique values from the person_home_ownership column in the data dataset, providing a quick overview of the different home ownership types recorded.

Handling Invalid Value by Data Correction:

R code:

```
47
48  ownership_corrections <- c("RENTT" = "RENT", "OOWN" = "OWN")
49  data <- data %>%
50    mutate(person_home_ownership = toupper(person_home_ownership)) %>%
51    mutate(person_home_ownership = case_when(
52      person_home_ownership %in% names(ownership_corrections) ~ ownership_corrections[person_home_ownership],
53      person_home_ownership %in% c("RENT", "OWN", "MORTGAGE", "OTHER") ~ person_home_ownership,
54      TRUE ~ "UNKNOWN"
55    ))
56  print(unique(data$person_home_ownership))
57
```

Output:

```
R 4.4.1 · C:/Users/F. M SHARIAR/Desktop/
> ownership_corrections <- c("RENTT" = "RENT", "OOWN" = "OWN")
> data <- data %>%
+   mutate(person_home_ownership = toupper(person_home_ownership)) %>%
+   mutate(person_home_ownership = case_when(
+     person_home_ownership %in% names(ownership_corrections) ~ ownership_corrections[person_home_ownership],
+     person_home_ownership %in% c("RENT", "OWN", "MORTGAGE", "OTHER") ~ person_home_ownership,
+     TRUE ~ "UNKNOWN"
+   ))
> print(unique(data$person_home_ownership))
[1] "RENT"     "OWN"      "MORTGAGE" "OTHER"
>
```

Description:

Standardizes the person_home_ownership column in the data dataset by converting all entries to uppercase and correcting specific mislabelled entries using predefined mappings. It checks for valid categories ("RENT", "OWN", "MORTGAGE", "OTHER") and assigns "UNKNOWN" to any outliers and then prints the unique values in the updated column to confirm the applied corrections and standardization.

Convert Loan Status to a Factor:

R code:

```r

data$loan_status <- as.factor(data$loan_status)

```

Description:

Converts the loan_status column in the data dataset to a factor, which is useful for statistical modeling and analysis that requires categorical inputs.

Identify Minority and Majority Classes:

R code:

```r

class_counts <- table(data$loan_status)
minority_class <- names(which.min(class_counts))
majority_class <- names(which.max(class_counts))

```

Description:

Calculates the frequency of each category within the loan_status column of the data dataset and identifies the minority and majority classes. It uses the table() function to count occurrences of each class in loan_status, then determines the class with the fewest instances (minority_class) and the one with the most instances (majority_class) by applying the which.min() and which.max() functions respectively to the counts.

Handling Imbalance Data (Oversampling):

R code:

```r

minority_data <- data %>% filter(loan_status == minority_class)
oversampled_minority_data <- minority_data %>% slice_sample(n = class_counts[majority_class], replace = TRUE)
balanced_data <- bind_rows(oversampled_minority_data, data %>% filter(loan_status == majority_class))

```

Description:

Addresses class imbalance by oversampling the minority class in the data dataset. First, it filters rows where loan_status matches the minority class, then oversamples these rows to equal the frequency of the majority class using slice_sample() with replacement. Finally, it combines these oversampled rows with those from the majority class to create balanced_data, a new dataset with an equal number of instances for each class.

Handling Imbalance Data (Undersampling):

R code:

```r

majority_data <- data %>% filter(loan_status == majority_class)
undersampled_majority_data <- majority_data %>% slice_sample(n = class_counts[minority_class])
balanced_data_undersample <- bind_rows(undersampled_majority_data, data %>% filter(loan_status == minority_class))

```

Description:

Addresses class imbalance by undersampling the majority class in the data dataset. It first isolates rows where loan_status matches the majority class, then randomly selects a subset equal to the number of minority class instances using slice_sample(). These rows are combined with those of the minority class to form balanced_data_undersample, a new dataset that ensures equal representation of both classes.

Print Oversampling Data:

R code:

```r

print(table(balanced_data$loan_status))

```

Description:

Displays the frequency of each category in the loan_status column of the balanced_data dataset, allowing verification of successful class balancing for further analysis.

Print Undersampling Data:

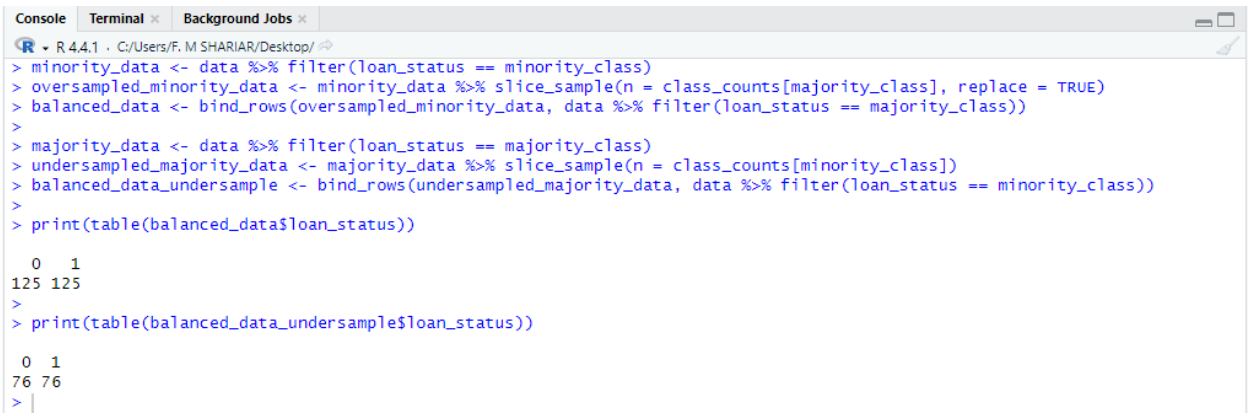
R code:

```r

print(table(balanced_data_undersample$loan_status))

```

Output:



Description:

Displays the class frequencies in balanced_data_undersample to confirm successful class balancing through undersampling.

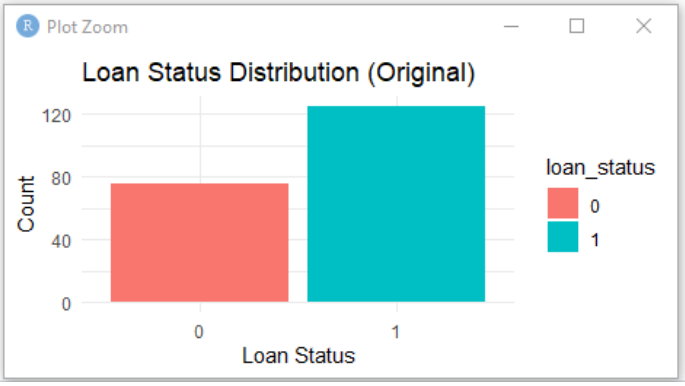
Bar Plot Data Distribution:

R code:

```
74
75  ggplot(data, aes(x = loan_status, fill = loan_status)) +
76    geom_bar() +
77    labs(title = "Loan Status Distribution (Original)", x = "Loan Status", y = "Count") +
78    theme_minimal()
79
```

Output:

```
> ggplot(data, aes(x = loan_status, fill = loan_status)) +
+   geom_bar() +
+   labs(title = "Loan Status Distribution (Original)", x = "Loan Status", y = "Count") +
+   theme_minimal()
> |
```



Description:

Create a bar plot showing the distribution of loan_status in the original data dataset. It maps loan_status to the x-axis and fill color, adds a title and axis labels, and applies a minimal theme for clarity. The plot highlights the frequency of each class, making it easy to identify class imbalances.

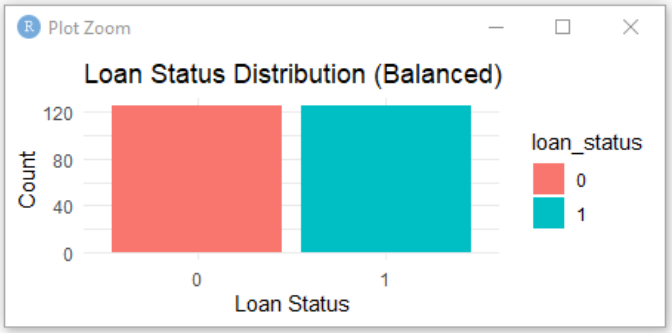
Bar plot Data Distribution After Oversampling:

R code:

```
79
80  ggplot(balanced_data, aes(x = loan_status, fill = loan_status)) +
81    geom_bar() +
82    labs(title = "Loan Status Distribution (Balanced)", x = "Loan Status", y = "Count") +
83    theme_minimal()
84
```

Output:

```
> ggplot(balanced_data, aes(x = loan_status, fill = loan_status)) +
+   geom_bar() +
+   labs(title = "Loan Status Distribution (Balanced)", x = "Loan Status", y = "Count") +
+   theme_minimal()
> |
```



Description:

Create a bar plot displaying the distribution of loan_status in the balanced_data dataset. It maps loan_status to the x-axis and fill color, adds a title ("Loan Status Distribution (Balanced)") and axis labels, and applies a minimal theme. The plot visually confirms that the dataset has been balanced, with equal counts for each class.

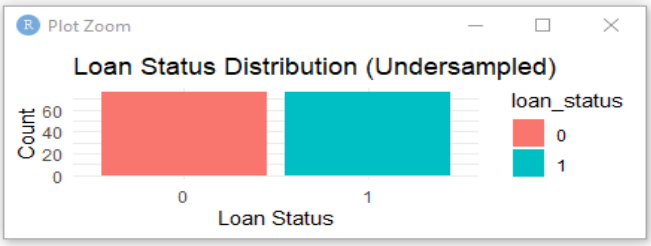
Bar plot Data Distribution After Undersampling:

R code:

```
84
85  ggplot(balanced_data_undersample, aes(x = loan_status, fill = loan_status)) +
86    geom_bar() +
87    labs(title = "Loan Status Distribution (Undersampled)", x = "Loan Status", y = "Count") +
88    theme_minimal()
89
```

Output:

```
> ggplot(balanced_data_undersample, aes(x = loan_status, fill = loan_status)) +
+   geom_bar() +
+   labs(title = "Loan Status Distribution (Undersampled)", x = "Loan Status", y = "Count") +
+   theme_minimal()
> |
```



Description:

Create a bar plot illustrating the distribution of loan_status in the balanced_data_undersample dataset. It maps loan_status to the x-axis and fill color, adds a title ("Loan Status Distribution (Undersampled)") and axis labels, and applies a minimal theme. The plot confirms that the dataset has been balanced through undersampling, showing equal counts for each class.

Check and Remove Duplicates:

R code:

```
89
90  data <- data %>% distinct()
91
```

Description:

Removes duplicate rows from the data dataset, ensuring all entries are unique and cleaning the data for analysis.

Detect and Handle Outliers:

R code:

```
92  numeric_columns <- sapply(data, is.numeric)
93  data[, numeric_columns] <- lapply(data[, numeric_columns], function(x) {
94    qnt <- quantile(x, probs=c(.25, .75), na.rm = TRUE)
95    caps <- quantile(x, probs=c(.05, .95), na.rm = TRUE)
96    H <- 1.5 * IQR(x, na.rm = TRUE)
97    x[x < (qnt[1] - H)] <- caps[1]
98    x[x > (qnt[2] + H)] <- caps[2]
99    x
100 })
101
```

Description:

It calculates the interquartile range (IQR) to identify outliers, which are values far below the 25th percentile or far above the 75th percentile. Any outliers are replaced with the 5th percentile (for very low values) or the 95th percentile (for very high values), ensuring the data remains consistent without extreme values. The updated columns are then saved back into the dataset, reducing the impact of outliers while preserving the overall structure.

Normalize Person Income:

R code:

```
101
102 data$person_income <- (data$person_income - min(data$person_income, na.rm = TRUE)) /
103   (max(data$person_income, na.rm = TRUE) - min(data$person_income, na.rm = TRUE))
104
```

Output:

```
> data$person_income <- (data$person_income - min(data$person_income, na.rm = TRUE)) /
+   (max(data$person_income, na.rm = TRUE) - min(data$person_income, na.rm = TRUE))
> data$person_income
  [1] 0.1676797824 0.0000000000 0.0004384079 0.1896142291 0.1513434673 0.0018800954 0.2051580376 0.2340086501
  [9] 0.2484367667 0.0012843103 0.2549032833 0.0023353652 0.2882756799 0.3328274781 0.9024608454 0.2051580376
 [17] 0.3728181478 0.2784648979 0.3494504445 0.0056234245 0.5155114899 0.4314102402 0.1882146962 0.0044515264
 [25] 0.2392639244 0.1971739552 0.0056374760 0.0056515275 0.1876498245 0.0076046910 0.2051580376 0.3725708408
 [33] 0.2876714639 1.0000000000 0.9802182484 0.0080599607 0.1309209657 0.1847608288 0.2051580376 0.2094493765
 [41] 0.1776704240 0.0078688598 0.9808280851 0.9815419031 0.9791053668 0.9799400280 0.9806903800 0.2415402731
 [49] 0.1926634123 0.2688761301 0.2314765634 0.2311983430 0.2778578715 0.2949923138 0.3729361807 0.9129310660
 [57] 0.9029067006 0.9024373793 0.4004996726 0.0082819750 0.4104622112 0.8957910031 0.8548504495 0.7177215154
 [65] 0.2048966791 0.0076749486 0.8453909559 0.8421647233 0.8279586210 0.8117684419 0.0080936844 0.1595130300
 [73] 0.0085011789 0.1880123541 0.2539730716 0.0082819750 0.2649669929 0.0083466120 0.0088524673 0.2474475386
 [81] 0.8112485351 0.8103070823 0.3057052044 0.3403000846 0.0090576197 0.3332181107 0.4778477544 0.2011364882
 [89] 0.0087456756 0.2050428150 0.7774011966 0.2158091015 0.2479449629 0.2154606234 0.1787692541 0.3732368836
 [97] 0.0102463796 0.1710043756 0.7565880624 0.7573580865 0.7574058617 0.7132784199 0.7514255283 0.7439866454
[105] 0.2486391088 0.0110613687 0.0110416965 0.1114005727 0.1068675474 0.0115110178 0.0120562174 0.7304522065
[113] 0.7279875672 0.6873449062 0.7280381527 0.0117695661 0.0121320957 0.1105181363 0.1299317376 0.0115587930
[121] 0.0129498950 0.1109087690 0.1337200316 0.7269983391 0.7267116878 0.7263154345 0.7267313599 0.1355101972
[129] 0.1362212049 0.1364319779 0.1167485871 0.1363532893 0.1362015327 0.1369940393 0.1431963871 0.0136777646
```

Description:

Normalizes the person_income column in data to a 0–1 scale by subtracting the minimum value and dividing by the range (max - min), ensuring all values are comparable and on the same scale.

Transform Attribute:

R code:

```
104
105 data$loan_category <- cut(data$loan_amnt, breaks=c(0, 5000, 20000, Inf), labels=c("Low", "Medium", "High"))
106 data$person_gender_numeric <- as.numeric(factor(data$person_gender))
107
```

Output:

```
> data$loan_category <- cut(data$loan_amnt, breaks=c(0, 5000, 20000, Inf), labels=c("Low", "Medium", "High"))
> data$person_gender_numeric <- as.numeric(factor(data$person_gender))
> data$loan_category
  [1] High   Low    Medium High   High   Low    High   High   High   Low    High   Low    High   High   High   High
 [17] High   High   High   Low    High   High   High   Low    High   High   Low    Low    High   Low    Low    High
 [33] High   High   Medium Low    High   High   High   High   High   Low    Medium High   Medium High   Medium High
 [49] High   High   High   High   High   High   High   Medium Medium High   High   Low    High   Medium Medium High
 [65] High   Low    High   Medium Medium High   Low    High   Low    High   High   Medium High   Low    Low    High
 [81] Medium Low    High   High   Low    High   High   High   Low    High   High   High   High   High   High   High
 [97] Low    High   Medium Medium High   Medium High   High   High   Low    Low    High   High   Low    Low    Medium
[113] High   High   Medium Low    Low    High   High   Low    Low    High   High   High   Low    Medium High   High
[129] High   High   High   High   High   High   High   Low    High   High   High   Medium High   Medium High   High
[145] High   High   High   Low    Low    Medium High   High   High   High   High   High   High   High   Medium High
[161] Medium High   High   High   Low    Medium Medium Medium High   High   High   High   High   High   High   High
[177] High   Medium Medium Medium High   High   Medium Medium Medium Medium High   Medium High   Low    High   High
[193] High   High   High   Medium Medium Low    High   Low
Levels: Low Medium High
```

Description:

Adds two new columns to data: loan_category classifies loan_amnt into "Low," "Medium," and "High" based on ranges, and person_gender_numeric converts person_gender into numeric codes for analysis.

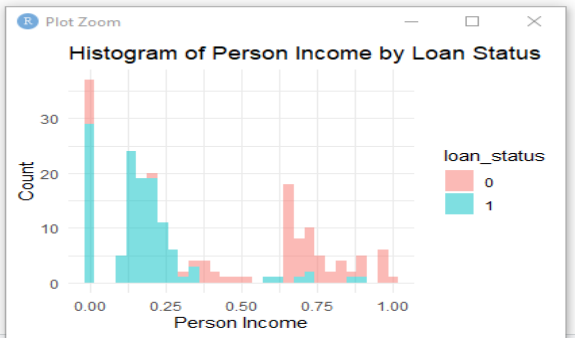
Histogram of Person Income by Loan Status:

R code:

```
107
108 ggplot(data, aes(x = person_income, fill = loan_status)) + geom_histogram(bins = 30, alpha = 0.5) +
109   labs(title = "Histogram of Person Income by Loan Status", x = "Person Income", y = "Count") +
110   theme_minimal()
111
```

Output:

```
> ggplot(data, aes(x = person_income, fill = loan_status)) + geom_histogram(bins = 30, alpha = 0.5) +
+   labs(title = "Histogram of Person Income by Loan Status", x = "Person Income", y = "Count") +
+   theme_minimal()
> |
```



Description:

Create a histogram showing the distribution of person_income by loan_status nd helps to visualize income patterns related to loan status.

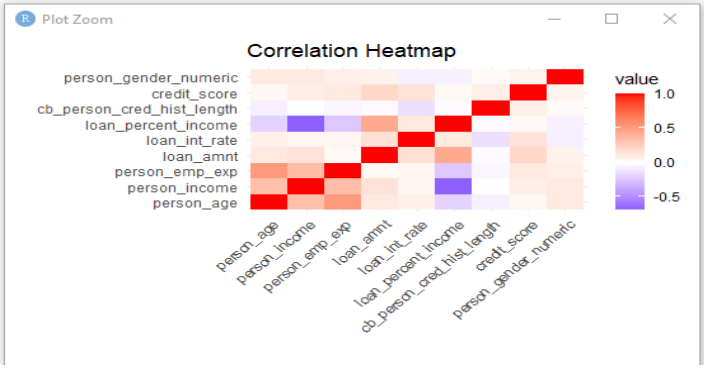
Correlation Heatmap:

R code:

```
111
112 numeric_data <- data %>% select(where(is.numeric))
113 cor_matrix <- cor(numeric_data, use = "complete.obs")
114 cor_melted <- melt(cor_matrix)
115 ggplot(data = cor_melted, aes(x = Var1, y = Var2, fill = value)) +
116   geom_tile() +
117   scale_fill_gradient2(low = "blue", high = "red", mid = "white", midpoint = 0) +
118   labs(title = "Correlation Heatmap", x = "", y = "") +
119   theme_minimal() +
120   theme(axis.text.x = element_text(angle = 45, hjust = 1))
121
```

Output:

```
> numeric_data <- data %>% select(where(is.numeric))
> cor_matrix <- cor(numeric_data, use = "complete.obs")
> cor_melted <- melt(cor_matrix)
> ggplot(data = cor_melted, aes(x = Var1, y = Var2, fill = value)) +
+   geom_tile() +
+   scale_fill_gradient2(low = "blue", high = "red", mid = "white", midpoint = 0) +
+   labs(title = "Correlation Heatmap", x = "", y = "") +
+   theme_minimal() +
+   theme(axis.text.x = element_text(angle = 45, hjust = 1))
> |
```



Description:

Creates a correlation heatmap to visualize relationships between numeric variables in the data dataset. It calculates pairwise correlations, melts the matrix into a plottable format, and uses ggplot2 to map correlations to colors—blue for negative, red for positive, and white for neutral.

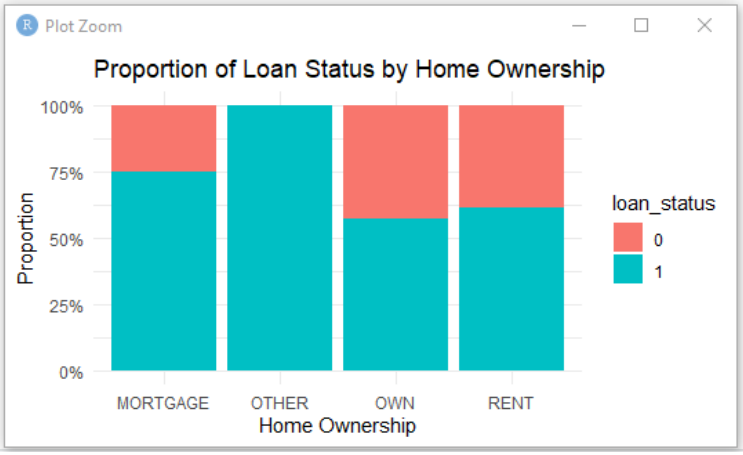
Proportion of Loan Status by Home Ownership:

R code:

```
121
122 ggplot(data, aes(x = person_home_ownership, fill = loan_status)) +
123   geom_bar(position = "fill") +
124   labs(title = "Proportion of Loan Status by Home Ownership", x = "Home Ownership", y = "Proportion") +
125   scale_y_continuous(labels = scales::percent) +
126   theme_minimal()
127
```

Output:

```
> ggplot(data, aes(x = person_home_ownership, fill = loan_status)) +
+   geom_bar(position = "fill") +
+   labs(title = "Proportion of Loan Status by Home Ownership", x = "Home Ownership", y = "Proportion") +
+   scale_y_continuous(labels = scales::percent) +
+   theme_minimal()
> |
```



Description:

Creates a proportional bar plot to show how loan_status varies across different person_home_ownership categories.

Save the Dataset in a .csv File:

R code:

```
127
128 write.csv(data, "C:/Users/F. M SHARIAR/Desktop/9th Semester/Data Science/Mid/Project/final_processed_data.csv", row.names = FALSE)
129
```

Description:

Saves the data dataset to a CSV file in the specified directory. The argument row.names = FALSE ensures that row numbers are not included in the file, keeping the output clean and focused on the dataset's content.

Final Structure of the Dataset:

R code:

```
129
130 str(data)
131
```

Output:

```
> str(data)
tibble [200 × 16] (S3: tbl_df/tbl/data.frame)
 $ person_age                    : num [1:200] 21 21 25 23 24 23 22 24 22 21 ...
 $ person_gender                 : chr [1:200] "female" "female" "female" "female" ...
 $ person_education              : chr [1:200] "Master" "High School" "High School" "Bachelor" ...
 $ person_income                 : num [1:200] 0.16768 0 0.000438 0.189614 0.151343 ...
 $ person_emp_exp                : num [1:200] 0 0 3 0 1 0 1 5 3 0 ...
 $ person_home_ownership         : Named chr [1:200] "RENT" "OWN" "MORTGAGE" "RENT" ...
  ..- attr(*, "names")= chr [1:200] "" "" "" "" ...
 $ loan_amnt                     : num [1:200] 35000 1000 5500 35000 35000 2500 35000 35000 35000 1600 ...
 $ loan_intent                   : chr [1:200] "PERSONAL" "EDUCATION" "MEDICAL" "MEDICAL" ...
 $ loan_int_rate                 : num [1:200] 16 11.1 12.9 15.2 14.3 ...
 $ loan_percent_income           : num [1:200] 0.49 0.235 0 0.44 0.53 0.19 0.37 0.37 0.35 0.13 ...
 $ cb_person_cred_hist_length    : num [1:200] 3 2 3 2 4 2 3 4 2 3 ...
 $ credit_score                  : num [1:200] 561 504 635 675 586 532 701 585 544 640 ...
 $ previous_loan_defaults_on_file: chr [1:200] "No" "Yes" "No" "No" ...
 $ loan_status                   : Factor w/ 2 levels "0","1": 2 1 2 2 2 2 2 2 2 2 ...
 $ loan_category                 : Factor w/ 3 levels "Low","Medium",..: 3 1 2 3 3 1 3 3 3 1 ...
 $ person_gender_numeric         : num [1:200] 1 1 1 1 2 1 1 2 1 1 ...
>
```

Description:

Provides a quick summary of the data dataset, showing its structure, the number of rows and columns, column names, data types, and a preview of the first few values in each column. It helps verify the dataset's format for analysis.

Final Summary of the Dataset:

R code:

```
131
132  summary(data)
133
```

Output:

```
> summary(data)
   person_age    person_gender      person_education   person_income    person_emp_exp  person_home_ownership
 Min.   :21.00   Length:200         Length:200         Min.   :0.0000   Min.   :0.000   Length:200
 1st Qu.:22.00   Class :character   Class :character   1st Qu.:0.1360   1st Qu.:0.000   Class :character
 Median :23.00   Mode  :character   Mode  :character   Median :0.2052   Median :1.000   Mode  :character
 Mean   :23.51                                        Mean   :0.3446   Mean   :1.575
 3rd Qu.:25.00                                        3rd Qu.:0.6429   3rd Qu.:3.000
 Max.   :26.00                                        Max.   :1.0000   Max.   :7.000
   loan_amnt     loan_intent        loan_int_rate   loan_percent_income cb_person_cred_hist_length  credit_score
 Min.   : 1000   Length:200         Min.   : 5.42   Min.   :0.0000      Min.   :2.00               Min.   :504.0
 1st Qu.:10000   Class :character   1st Qu.:10.65   1st Qu.:0.0900      1st Qu.:2.00               1st Qu.:594.8
 Median :25000   Mode  :character   Median :11.85   Median :0.2325      Median :3.00               Median :629.0
 Mean   :20493                      Mean   :12.30   Mean   :0.2285      Mean   :2.99               Mean   :627.5
 3rd Qu.:28000                      3rd Qu.:14.45   3rd Qu.:0.3400      3rd Qu.:4.00               3rd Qu.:664.2
 Max.   :35000                      Max.   :20.00   Max.   :0.5300      Max.   :4.00               Max.   :718.0
 previous_loan_defaults_on_file loan_status loan_category person_gender_numeric
 Length:200                     0: 76       Low   : 38    Min.   :1.000
 Class :character               1:124       Medium: 37    1st Qu.:1.000
 Mode  :character                           High  :125    Median :2.000
                                                          Mean   :1.615
```

Description:

Provides a statistical summary of each column in the data dataset. For numeric columns, it shows metrics like minimum, maximum, mean, median, and quartiles. For categorical columns, it displays the frequency of each category.